



Facultad de Estudios Superiores Acatlán

Diplomado en Ciencia de Datos

Proyecto final · Módulo V



Señal de arrepentimiento temprano
antes de comprar un videojuego

Fernando Axel Ramírez Gómez

30 de septiembre de 2026

Repositorio: <https://github.com/fernandoaxelramirezgomez-coder/nexplay>

App en vivo: <https://nexplay-six.vercel.app>



Resumen

[PENDIENTE: redacción]



Índice

Resumen	1
1. Introducción	5
1.1. Contexto	5
1.2. Preguntas del proyecto	5
1.3. Qué entrega el proyecto	5
1.4. Mapa del documento contra la rúbrica	5
2. Planteamiento del problema	5
2.1. La decisión antes de pagar	5
2.2. Por qué el promedio de reseñas no basta	5
2.3. Qué se puede observar y qué no	5
3. Nicho de negocio y usuario final	5
3.1. Una capa previa a la compra	5
3.2. El usuario, por comportamiento	6
3.3. Veteranos y novatos	6
4. Estrategia: accionables, usuarios y usabilidad	6
4.1. Accionables	6
4.2. Usuarios y necesidades	6
4.3. Recorrido: PAYDAY 3 contra Dead Space	6
4.4. Usabilidad	6
5. Descripción de los datos	6
5.1. La API de Steam	6
5.2. Releases y su verificación	7
5.3. Periodo que cubren	7
5.4. Calidad	8
5.5. Por qué estos datos sirven para la pregunta	10
6. Definición del objetivo	10
6.1. La fórmula	10
6.2. Por qué es una señal proxy	10
6.3. Robustez del umbral de 120 minutos	11



6.4. Por qué PR-AUC	11
7. Análisis exploratorio	12
7.1. La tasa varía por juego	12
7.2. La crítica acompaña a la señal	13
7.3. El texto de las negativas tempranas	13
7.4. Los 83 contra los 40	13
8. Procesamiento de datos	13
8.1. Validación	13
8.2. Limpieza: marcar, no borrar	13
8.3. Idioma	13
8.4. Precio imputado	13
8.5. Variables no disponibles al comprar	13
9. Modelación	13
9.1. Clasificador trivial	13
9.2. Conjuntos completo, compra y juego	14
9.3. La decisión B+: no forzar una personalización que los datos no justifican	14
9.4. Validación agrupada por juego	14
9.5. Bootstrap de coeficientes	14
9.6. Umbrales de riesgo	14
9.7. Modelos de texto (Parte A)	14
10. Evaluación y generalización	14
10.1. Resultados por pliegue	14
10.2. Prueba externa en 40 títulos	14
10.3. Por qué cae fuera de muestra	14
10.4. Casos al filo del corte	14
11. Interpretabilidad	15
11.1. Factores por aporte y su evidencia	15
11.2. Lectura contra el catálogo	15
11.3. Lo típico del catálogo: una decisión de producto	15
11.4. Avisos	15
11.5. Los motivos: una heurística de palabras clave	15



12. Arquitectura y despliegue	15
12.1. De los datos a la interfaz	15
12.2. Verificaciones en el build	15
12.3. Endpoints	15
12.4. Costos y límites	15
13. Interfaz de uso	16
13.1. Las cinco vistas	16
13.2. Paso a paso de una consulta	16
13.3. Nía	16
13.4. Pruebas	16
14. Resolución del problema	16
15. Limitaciones	16
16. Conclusiones	16
17. Siguiendo pasos	16
Referencias	17
A. Reproducibilidad	17

1. Introducción

1.1. Contexto

[PENDIENTE: redacción]

1.2. Preguntas del proyecto

[PENDIENTE: redacción]

1.3. Qué entrega el proyecto

[PENDIENTE: redacción]

1.4. Mapa del documento contra la rúbrica

[PENDIENTE: redacción]

2. Planteamiento del problema

2.1. La decisión antes de pagar

[PENDIENTE: redacción]

2.2. Por qué el promedio de reseñas no basta

[PENDIENTE: redacción]

2.3. Qué se puede observar y qué no

[PENDIENTE: redacción]

3. Nicho de negocio y usuario final

3.1. Una capa previa a la compra

[PENDIENTE: redacción]

3.2. El usuario, por comportamiento

[PENDIENTE: redacción]

3.3. Veteranos y novatos

[PENDIENTE: redacción]

4. Estrategia: accionables, usuarios y usabilidad

4.1. Accionables

[PENDIENTE: redacción]

4.2. Usuarios y necesidades

[PENDIENTE: redacción]

4.3. Recorrido: PAYDAY 3 contra Dead Space

[PENDIENTE: redacción]

4.4. Usabilidad

[PENDIENTE: redacción]

5. Descripción de los datos

Los datos son reseñas públicas de Steam y la ficha de cada juego en la misma tienda. En cada reseña, Steam guarda si el voto fue negativo y cuántos minutos llevaba jugados quien la escribió; con esas dos cosas se define la señal de arrepentimiento temprano (sección 6). El modelo se entrena con 83 juegos y 123,972 reseñas (data-v1), y el sitio sirve un catálogo de 123 juegos y 184,367 reseñas (data-v3). No se usó Kaggle ni ningún conjunto de terceros.

5.1. La API de Steam

La ingesta, el proceso que bajó los datos de Steam, usa dos servicios públicos de la tienda¹:

¹Una API (interfaz de programación) es una dirección web que devuelve datos en un formato que un programa puede leer, en lugar de una página pensada para personas. Estos dos servicios no piden clave.

- `appreviews`, por juego, da el voto, los minutos jugados al escribir y en total, el texto, las fechas y dos datos del autor: cuántos juegos tiene y cuántas reseñas ha escrito. Se piden las reseñas más recientes (`filter=recent`), en inglés (`language=english`) y de cualquier forma de compra, de 100 en 100, hasta un tope de 1,500 por juego. Valve documenta este servicio en Valve Corporation (s.f.-b). Al consultarla para este documento, la página lo marca como obsoleto y remite a `GetAppReviews` (Valve Corporation, s.f.-a). El proyecto no depende de volver a bajarlo: los datos quedaron congelados en releases (sección 5.2).
- `appdetails`, por juego, da el precio y el descuento en pesos mexicanos (`cc=mx`), los géneros, la fecha de lanzamiento, la descripción en español (`l=spanish`) y la nota de Metacritic². Valve no documenta este servicio de forma oficial.

La nota de Metacritic se toma de la tienda; el proyecto nunca consulta Metacritic. En los 40 títulos externos se comparó la nota guardada con la que mostraba la tienda el 21 de septiembre de 2026, y coinciden 40 de 40³. Cuando la tienda no muestra nota, esa ausencia se guarda tal cual y el modelo la usa como una variable más.

5.2. Releases y su verificación

Cada corte de datos se publica como un release de GitHub con un tag fijo⁴. Antes de usar un archivo, se compara su sha256 con el publicado⁵. La tabla 1 resume los tres releases. Con `data-v1` se toman todas las decisiones del modelo: las variables, el entrenamiento y los umbrales. El release `data-v2` suma 40 juegos que el modelo nunca ve, y `data-v3` es lo que sirve el sitio.

Ningún release guarda el identificador de Steam de quien escribió la reseña.

5.3. Periodo que cubren

Las reseñas de `data-v1` se bajaron el 14 de septiembre de 2026, y las de los juegos que agregó `data-v2`, hasta el 21 de septiembre de 2026, en UTC⁶. La ingesta pide las reseñas más recientes de cada juego hasta llegar al tope, y 81 de los 83 juegos lo alcanzan. Un juego popular llena el tope en pocos días y uno de nicho necesita años: la muestra de cada juego cubre de 4 a 1,252 días, con una mediana de 64.

La figura 1 cuenta las reseñas de `data-v1` por mes de publicación. Casi todas las barras se juntan a la derecha: el 96.1 % de las reseñas de entrenamiento es de julio de 2025 en

²Metacritic es un sitio que promedia las calificaciones de la crítica especializada en una nota de 0 a 100. Steam muestra esa nota en la página de algunos juegos.

³La comparación está en `docs/evidencia/verificacion-40-steam.csv`.

⁴Un release es una versión publicada del repositorio, con archivos adjuntos. Su tag es el nombre que la identifica, como `data-v1`, y no cambia: quien baje `data-v1` dentro de un año recibe exactamente los mismos archivos.

⁵El sha256 es una huella de 64 caracteres que se calcula sobre el contenido de un archivo; si cambia un solo byte, cambia la huella. La comparación la hace `descargar_verificado`, en `backend/despliegue/utilidades.py`.

⁶UTC es el tiempo universal coordinado, la hora de referencia mundial. La Ciudad de México va seis horas atrás.

Release	Publicado	Juegos	Reseñas	Base (MB)	Extracto (MB)	Para qué
data-v1	2026-09-16	83	123,972	11.4	1.4	Entrenar, elegir las variables y fijar los umbrales.
data-v2	2026-09-21	123	184,367	16.7	1.9	Suma 40 juegos que el modelo no ve: la prueba externa.
data-v3	2026-09-27	123	184,367	16.7	—	data-v2 más los totales públicos de Steam por juego. Lo sirve la API.

Tabla 1: Releases de datos. La base es la copia reproducible de la base de datos (`nexplay_reproducible.db.xz`); el extracto, la tabla sin texto que lee el notebook 01 (`nexplay_extracto.parquet`). Los sha256 completos están en el anexo. Fuente: API de releases de GitHub y los archivos bajados.

adelante (97.1 % en el catálogo), y a la izquierda queda una cola delgada que llega hasta el 8 de abril de 2023.



Figura 1: Reseñas de data-v1 por mes de publicación. Fuente: release data-v1, columna `timestamp_created`.

La muestra retrata, entonces, el presente de cada juego. En un juego viejo, la señal temprana viene de quien lo compra tarde, en una oferta o después de una actualización. Esto es una limitación de la fuente (sección 15) y no se corrige con pesos, porque no hay forma de saber cómo era la mezcla de compradores en otra época.

5.4. Calidad

Antes de explorar, el notebook `00_exploracion` valida data-v1: tipos y nulos, llaves, rangos, textos repetidos y la partición de validación. La tabla 2 separa los chequeos que no encontraron nada de los que sí, con lo que se hizo en cada caso.

Las dos columnas que definen la señal, el voto y los minutos al reseñar, no tienen nulos

Chequeo (data-v1)	Resultado	Qué se hizo
<i>Sin hallazgos</i>		
Nulos en las reseñas	0 en 21 columnas	nada que corregir
Llaves: reseña repetida, juego sin reseñas, reseña sin juego	0 casos	nada que corregir
Rangos imposibles (6 reglas: minutos negativos, voto fuera de 0 y 1, fechas, precio, descuento, nota)	0 casos	nada que corregir
Nota de Metacritic contra la de la tienda hoy (40 juegos externos)	40 de 40 iguales	nada que corregir
<i>Hallazgos y qué se hizo</i>		
Más minutos al reseñar que en total ^a	1 caso	se conserva: la señal no cambia con ninguno de los dos contadores
Juegos de pago sin precio ^b	2: Grand Theft Auto V Legacy, New World: Aeternum	precio tomado como 0, con aviso en la ficha (§8)
Juegos sin nota de Metacritic	23 de 83	la falta de nota es una variable del modelo ^c
Textos repetidos de 8 palabras o más	359 en el mismo juego; 57 en 2 o más juegos	se marcan, no se borran (§8)
Reseñas plantilla	192 (0.15 %)	se marcan, no se borran
Perfil privado: aparece con 0 juegos ^d	59.9 % de las reseñas	es una bandera de privacidad, no una biblioteca vacía

Columnas: ^a playtime_at_review y playtime_forever; ^b es_gratis y precio_final; ^c metacritic_disponible; ^d num_games_owned.

Tabla 2: Chequeos de calidad sobre data-v1. Fuente: las funciones de backend/analisis/exploracion.py que usa el bloque de validación del notebook 00.

ni valores fuera de rango, y todas las variables del juego que usa el modelo se conocen antes de comprar. Tres hallazgos tocan al modelo, y ninguno se rellena en silencio. El precio de los 2 juegos de pago que llegaron sin precio se toma como 0, y la ficha lo avisa. La falta de nota de los 23 juegos sin Metacritic entra al modelo como una variable propia. Y el 0 de los perfiles privados⁷, que son el 59.9 % de las reseñas, se trata como una bandera de privacidad y no como una biblioteca vacía. Los textos repetidos y las plantillas se marcan y se conservan; la sección 8 explica por qué.

El tope de 1,500 reseñas por juego es una decisión de diseño: así cada título pesa casi lo mismo en el entrenamiento, y unos cuantos juegos muy populares no dominan la muestra. Tiene otra consecuencia, los empates de tamaño entre juegos, que la sección 9.4 trata al hablar de la validación.

⁷En Steam, cada persona decide si su perfil es público. Un perfil privado oculta la biblioteca, y la API lo reporta con 0 juegos.

5.5. Por qué estos datos sirven para la pregunta

NexPlay quiere saber, antes de la compra, qué tan seguido un juego deja a sus compradores arrepentidos en las primeras horas. Estos datos sirven para eso. Las reseñas las escriben personas que tienen el juego, y cada una dice cuántos minutos llevaba jugados su autor al escribirla: con eso se sabe si cae dentro o fuera de la ventana de reembolso de dos horas. Por su lado, `appdetails` da lo que cualquiera ve en la tienda antes de pagar, como el precio, el descuento y la nota de la crítica, y de ahí salen las variables del modelo, todas conocidas antes de la compra.

También tienen límites, que la sección 15 desarrolla: las reseñas son en inglés y las escriben compradores de todo el mundo, mientras que los precios son los de la tienda de México y quien usa NexPlay compra desde México.

6. Definición del objetivo

El modelo aprende a ordenar juegos según una señal que sale de sus reseñas: un voto negativo escrito antes de las dos horas de juego. Esa señal no es el arrepentimiento mismo, y su corte de 120 minutos viene de una regla de Steam. Además es rara, y eso obliga a elegir bien la métrica.

6.1. La fórmula

Para cada reseña i ,

$$Y_i = \begin{cases} 1 & \text{si } \text{playtime_at_review}_i < 120 \text{ y } \text{voted_up}_i = 0, \\ 0 & \text{en otro caso,} \end{cases}$$

donde `playtime_at_review` son los minutos que llevaba jugados quien escribió la reseña cuando la publicó y `voted_up` es el pulgar: 1 arriba, 0 abajo⁸. En `data-v1` hay 2,714 reseñas con $Y = 1$ de 123,972, una prevalencia de 2.19 % por reseña⁹. A esa etiqueta se le llama *señal de arrepentimiento temprano*.

6.2. Por qué es una señal proxy

Steam no registra arrepentimiento. Registra que alguien votó en contra y cuánto había jugado al escribir, así que la señal es una proxy¹⁰. Los 120 minutos vienen de la política de reembolsos de la tienda: se puede pedir el reembolso dentro de las dos semanas posteriores a la compra y con menos de dos horas de juego (Valve Corporation, 2024). Una reseña negativa escrita dentro de esa ventana es lo más cercano que dejan los datos a «esto no era para mí, y me di cuenta pronto».

⁸ Al reseñar, Steam pide elegir entre «Recomendado» (pulgar arriba) y «No recomendado» (pulgar abajo).

⁹ La prevalencia es la proporción de casos positivos; aquí, de reseñas con $Y = 1$.

¹⁰ Una variable proxy es una medida que sí se puede observar y que se usa en lugar de otra que no se puede observar directamente.

Por eso el sitio y este documento la llaman señal de arrepentimiento temprano y nunca dicen que alguien se arrepintió. La señal deja fuera a quien se arrepiente y no escribe, a quien pide el reembolso sin reseñar y a quien reseña después de las dos horas. Tampoco mide abandono ni reembolsos: de ninguno de los dos hay dato.

Lo que sí muestran los datos es que las negativas llegan antes. El 15.6 % de las reseñas negativas se escribe antes de los 120 minutos, contra el 2.0 % de las positivas. La señal se queda con ese tramo temprano de las negativas.

6.3. Robustez del umbral de 120 minutos

El umbral no se eligió con los datos: sale de la regla de Steam. Lo que sí se revisó es si moverlo cambia qué juegos tienen más señal. Con 60, 90 o 180 minutos cambia cuántas reseñas cuentan, pero el orden de los juegos casi no se mueve. Su correlación de Spearman con el orden de 120 va de 0.96 a 0.99 (tabla 3)¹¹.

Umbral (min)	Reseñas con señal	Prevalencia por reseña	Spearman con 120 (por juego)
60	1,649	1.33 %	0.964
90	2,250	1.81 %	0.989
120	2,714	2.19 %	1.000
180	3,349	2.70 %	0.993

Tabla 3: La señal con otros umbrales. La prevalencia es por reseña; el Spearman compara las tasas por juego de los 83 juegos contra las de 120 minutos. Fuente: data-v1, con `sensibilidad_umbral` de `backend/analisis/exploracion.py`.

La figura 2 muestra en qué minuto de juego se escriben las reseñas, por separado para las positivas y las negativas. El eje horizontal está en escala logarítmica¹², y cada curva se normaliza sobre su propio grupo para poder compararlas.

En los 120 minutos no hay escalón. Por minuto, las negativas pasan de 11.9 a 12.2 al cruzarlos, y las positivas, de 17.4 a 18.6. El único salto está en los 180 minutos y solo en las positivas, que pasan de 17.6 a 81.7 reseñas por minuto en 81 juegos. Las negativas apenas cambian (de 9.9 a 12.4).

Ese salto no toca la señal, que se define en 120 minutos y con reseñas negativas, y su causa no está en estos datos.

6.4. Por qué PR-AUC

Con una prevalencia de 2.19 %, un modelo que siempre dice «no» acierta el 97.81 % de las veces y no encuentra un solo caso. La exactitud premia justo eso, así que aquí no sirve. El proyecto usa el área bajo la curva de precisión y exhaustividad (PR-AUC, con

¹¹La correlación de Spearman compara dos órdenes, no dos valores: vale 1 cuando los juegos quedan en exactamente el mismo orden con los dos umbrales, aunque sus tasas cambien.

¹²En una escala logarítmica, cada marca del eje multiplica a la anterior por la misma cantidad, por ejemplo por diez. Así caben en una sola gráfica las reseñas escritas a los diez minutos y las escritas después de miles de horas.

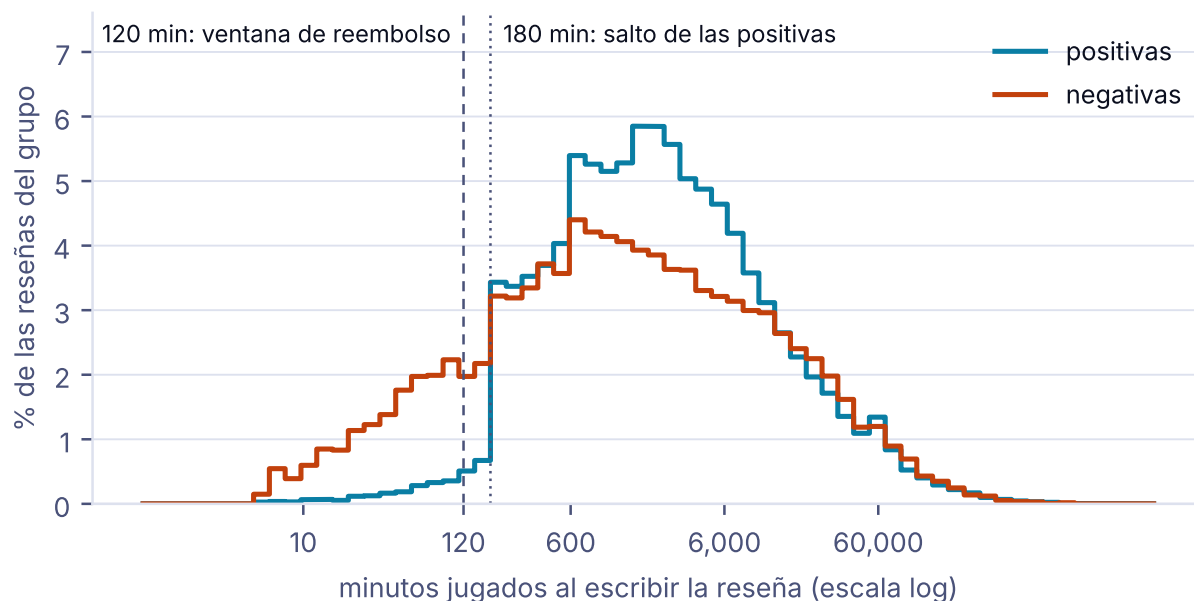


Figura 2: Minutos jugados al escribir la reseña, por pulgar, en escala logarítmica. Cada curva se normaliza sobre su propio grupo. Los intervalos tienen el mismo ancho en escala log, con un borde en 180 y ninguno en 120. Fuente: data-v1.

average_precision_score de scikit-learn)¹³. Mide qué proporción de lo que el modelo pone arriba es de verdad señal, y con clases desbalanceadas informa más que la curva ROC¹⁴ (Davis & Goadrich, 2006; Saito & Rehmsmeier, 2015).

El PR-AUC de un clasificador que no sabe nada es la prevalencia, 0.0219¹⁵. Ese es el piso, y por eso en este documento el PR-AUC del modelo siempre va junto al del trivial. En la validación agrupada por juego¹⁶ el modelo llega a 0.0694, 3.2 veces ese piso (sección 10).

7. Análisis exploratorio

7.1. La tasa varía por juego

[PENDIENTE: redacción]

¹³La precisión es la proporción de los casos que el modelo pone arriba que de verdad tienen la señal; la exhaustividad, la proporción de casos con señal que el modelo encuentra. La curva traza una contra la otra al mover el punto de corte, y el PR-AUC es el área bajo esa curva.

¹⁴La curva ROC compara los aciertos contra las falsas alarmas. Cuando los negativos son muchísimos, las falsas alarmas pesan poco en su cuenta, y la curva puede verse bien aunque el modelo señale muchos casos de más.

¹⁵Ese clasificador trivial ignora las variables y da a todos los casos la misma calificación, así que no ordena nada.

¹⁶La validación agrupada por juego (GroupKFold) reparte juegos completos entre entrenamiento y prueba: ninguna reseña de un juego de prueba se usa para entrenar. Así se mide qué tan bien funciona el modelo con juegos que no vio (sección 9.4).

7.2. La crítica acompaña a la señal

[PENDIENTE: redacción]

7.3. El texto de las negativas tempranas

[PENDIENTE: redacción]

7.4. Los 83 contra los 40

[PENDIENTE: redacción]

8. Procesamiento de datos

8.1. Validación

[PENDIENTE: redacción]

8.2. Limpieza: marcar, no borrar

[PENDIENTE: redacción]

8.3. Idioma

[PENDIENTE: redacción]

8.4. Precio imputado

[PENDIENTE: redacción]

8.5. Variables no disponibles al comprar

[PENDIENTE: redacción]

9. Modelación

9.1. Clasificador trivial

[PENDIENTE: redacción]

9.2. Conjuntos completo, compra y juego

[PENDIENTE: redacción]

9.3. La decisión B+: no forzar una personalización que los datos no justifican

[PENDIENTE: redacción]

9.4. Validación agrupada por juego

[PENDIENTE: redacción]

9.5. Bootstrap de coeficientes

[PENDIENTE: redacción]

9.6. Umbrales de riesgo

[PENDIENTE: redacción]

9.7. Modelos de texto (Parte A)

[PENDIENTE DE EVIDENCIA]

10. Evaluación y generalización

10.1. Resultados por pliegue

[PENDIENTE: redacción]

10.2. Prueba externa en 40 títulos

[PENDIENTE: redacción]

10.3. Por qué cae fuera de muestra

[PENDIENTE: redacción]

10.4. Casos al filo del corte

[PENDIENTE: redacción]

11. Interpretabilidad

11.1. Factores por aporte y su evidencia

[PENDIENTE: redacción]

11.2. Lectura contra el catálogo

[PENDIENTE: redacción]

11.3. Lo típico del catálogo: una decisión de producto

[PENDIENTE: redacción]

11.4. Avisos

[PENDIENTE: redacción]

11.5. Los motivos: una heurística de palabras clave

[PENDIENTE: redacción]

12. Arquitectura y despliegue

12.1. De los datos a la interfaz

[PENDIENTE: redacción]

12.2. Verificaciones en el build

[PENDIENTE: redacción]

12.3. Endpoints

[PENDIENTE: redacción]

12.4. Costos y límites

[PENDIENTE: redacción]

13. Interfaz de uso

13.1. Las cinco vistas

[PENDIENTE: redacción]

13.2. Paso a paso de una consulta

[PENDIENTE: redacción]

13.3. Nia

[PENDIENTE: redacción]

13.4. Pruebas

[PENDIENTE: redacción]

14. Resolución del problema

[PENDIENTE: redacción]

15. Limitaciones

[PENDIENTE: redacción]

16. Conclusiones

[PENDIENTE: redacción]

17. Siguiendo pasos

[PENDIENTE: redacción]

Referencias

- Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233-240. <https://doi.org/10.1145/1143844.1143874>
- Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Valve Corporation. (s.f.-a). *IUserReviewsService*. Consultado el 30 de septiembre de 2026, desde <https://partner.steamgames.com/doc/webapi/IUserReviewsService>
- Valve Corporation. (s.f.-b). *User Reviews – Get List* [Consultada el 30 de septiembre de 2026; la página marca el endpoint como obsoleto y remite a IUserReviewsService]. Consultado el 30 de septiembre de 2026, desde <https://partner.steamgames.com/doc/store/getreviews>
- Valve Corporation. (2024, 23 de abril). *Steam Refunds*. Consultado el 30 de septiembre de 2026, desde https://store.steampowered.com/steam_refunds/

A. Reproducibilidad

[PENDIENTE: redacción]